Introduction to 🤗 Diffusers

In this notebook, you'll train your first diffusion model to **generate images of cute butterflies 🦋.** Along the way, you'll learn about the core components of the 🤗 Diffusers library, which will provide a good foundation for the more advanced applications that we'll cover later in the course.

Let's dive in!

What You Will Learn

In this notebook you will:

- See a powerful custom diffusion model pipeline in action (with information on how to make your own version)
- Create your own mini pipeline by:
  - Recapping the core ideas behind diffusion models
  - Loading in data from the Hub for training
  - Exploring how we add noise to this data with a scheduler
  - Creating and training the UNet model
  - Putting the pieces together into a working pipeline
- Edit and run a script for initializing longer training runs, that will handle
  - Multi-GPU training via 🤗 Accelerate
  - Experiment logging to track critical stats
  - Uploading the final model to the Hugging Face Hub

❓If you have any questions, please post them on the `#diffusion-models-class` channel on the Hugging Face Discord server. If you haven't signed up yet, you can do so here: https://huggingface.co/join/discord

Prerequisites

Before diving into the notebook, you should:

- 📖 Read the Unit 1 materials
- 🤗 Create an account on the Hugging Face Hub. If you haven't done so yet, you can do so here: https://huggingface.co/join

Step 1: Setup

Run the following cell to install the diffusers library as well as a few other requirements:

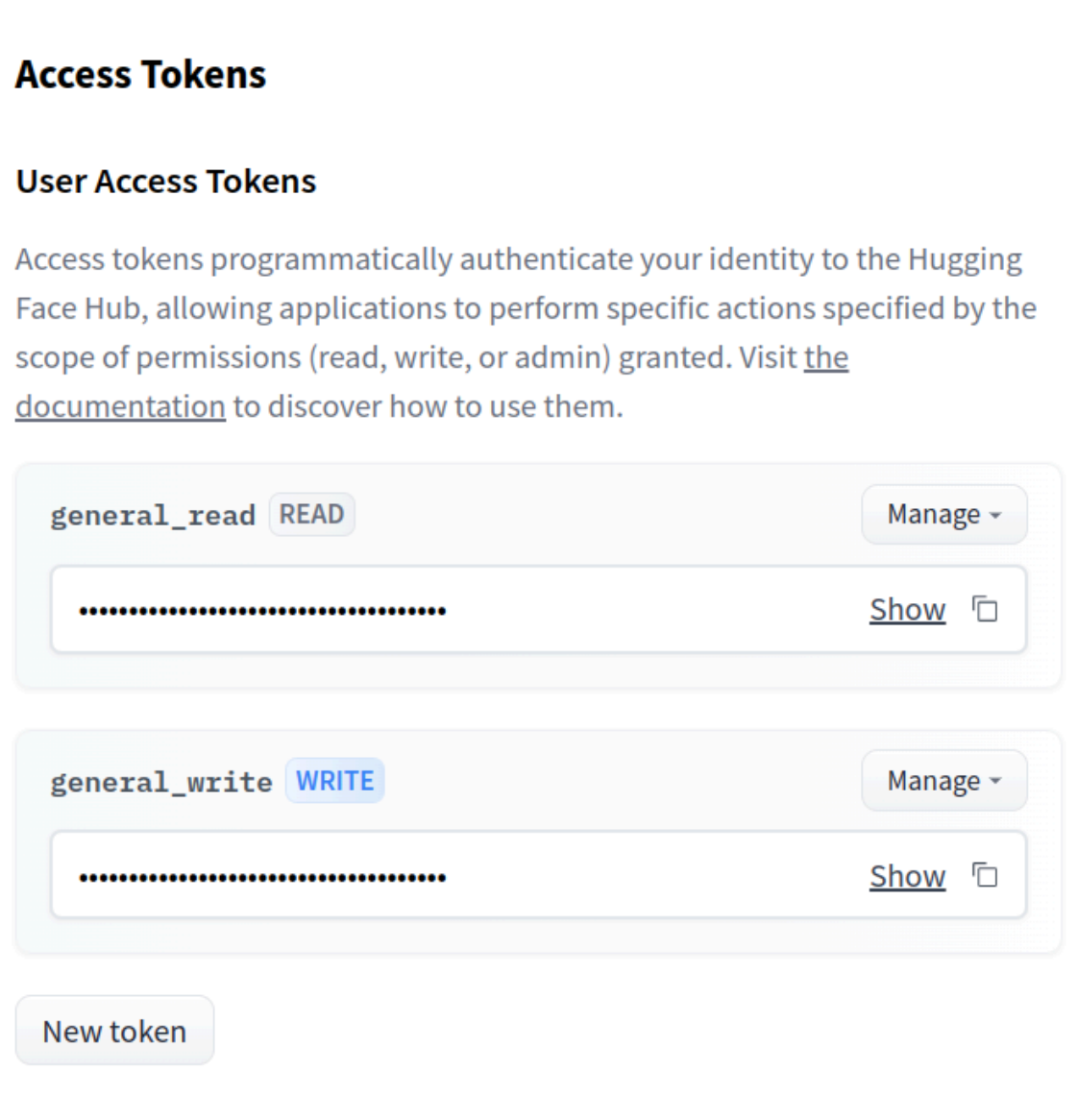
In [1]:
```
%pip install -qq -U diffusers datasets transformers accelerate ftfy pyarrow
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
%pip install ipywidgets
```

```
Collecting ipywidgets
  Downloading ipywidgets-8.1.8-py3-none-any.whl.metadata (2.4 kB)
Requirement already satisfied: comm>=0.1.3 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipywidgets) (0.2.3)
Requirement already satisfied: ipython>=6.1.0 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipywidgets) (9.9.0)
Requirement already satisfied: traitlets>=4.3.1 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipywidgets) (5.14.3)
Collecting widgetsnbextension~=4.0.14 (from ipywidgets)
  Downloading widgetsnbextension-4.0.15-py3-none-any.whl.metadata (1.6 kB)
Collecting jupyterlab_widgets~=3.0.15 (from ipywidgets)
  Downloading jupyterlab_widgets-3.0.16-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: colorama>=0.4.4 in c:\users\eraye\appdata\local\programs\python\python313\lib\site-packages (from ipython>=6.1.0->ipywidgets) (0.4.6)
Requirement already satisfied: decorator>=4.3.2 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipython>=6.1.0->ipywidgets) (5.2.1)
Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipython>=6.1.0->ipywidgets) (1.1.1)
Requirement already satisfied: jedi>=0.18.1 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipython>=6.1.0->ipywidgets) (0.19.2)
Requirement already satisfied: matplotlib-inline>=0.1.5 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipython>=6.1.0->ipywidgets) (0.2.1)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipython>=6.1.0->ipywidgets) (3.0.52)
Requirement already satisfied: pygments>=2.11.0 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipython>=6.1.0->ipywidgets) (2.19.2)
Requirement already satisfied: stack_data>=0.6.0 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from ipython>=6.1.0->ipywidgets) (0.6.3)
Requirement already satisfied: wcwidth in c:\users\eraye\appdata\roaming\python\python313\site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=6.1.0->ipywidgets) (0.3.0)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from jedi>=0.18.1->ipython>=6.1.0->ipywidgets) (0.8.5)
Requirement already satisfied: executing>=1.2.0 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in c:\users\eraye\appdata\roaming\python\python313\site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (3.0.1)
Requirement already satisfied: pure-eval in c:\users\eraye\appdata\roaming\python\python313\site-packages (from stack_data>=0.6.0->ipython>=6.1.0->ipywidgets) (0.2.3)
Downloading ipywidgets-8.1.8-py3-none-any.whl (139 kB)
Downloading jupyterlab_widgets-3.0.16-py3-none-any.whl (914 kB)
   ---------------------------------------- 0.0/914.9 kB ? eta -:--:--
   ----------- ---------------------------- 262.1/914.9 kB ? eta -:--:--
   ---------------------------------------- 914.9/914.9 kB 3.1 MB/s  0:00:00
Downloading widgetsnbextension-4.0.15-py3-none-any.whl (2.2 MB)
   ---------------------------------------- 0.0/2.2 MB ? eta -:--:--
   --------- ------------------------------ 0.5/2.2 MB 2.8 MB/s eta 0:00:01
   ------------------ --------------------- 1.0/2.2 MB 2.4 MB/s eta 0:00:01
   ----------------------- ---------------- 1.3/2.2 MB 2.4 MB/s eta 0:00:01
   --------------------------------- ------ 1.8/2.2 MB 2.3 MB/s eta 0:00:01
   ---------------------------------------- 2.2/2.2 MB 2.3 MB/s  0:00:00
Installing collected packages: widgetsnbextension, jupyterlab_widgets, ipywidgets

   -------------------------- ------------- 2/3 [ipywidgets]
   -------------------------- ------------- 2/3 [ipywidgets]
   -------------------------- ------------- 2/3 [ipywidgets]
   -------------------------- ------------- 2/3 [ipywidgets]
   -------------------------- ------------- 2/3 [ipywidgets]
   -------------------------- ------------- 2/3 [ipywidgets]
   ---------------------------------------- 3/3 [ipywidgets]

Successfully installed ipywidgets-8.1.8 jupyterlab_widgets-3.0.16 widgetsnbextension-4.0.15
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Next, head over to https://huggingface.co/settings/tokens and create an access token with write permission if you don't already have one:



Access Tokens

User Access Tokens

Access tokens programmatically authenticate your identity to the Hugging Face Hub, allowing applications to perform specific actions specified by the scope of permissions (read, write, or admin) granted. Visit the documentation to discover how to use them.

`general_read` READ — Manage ▾

•••••••••••••••••••••••••••••••••••••• Show

`general_write` WRITE — Manage ▾

•••••••••••••••••••••••••••••••••••••• Show

New token

You can login with this token using the command line (`huggingface-cli login`) or by running the following cell:

In [3]:
```
from huggingface_hub import notebook_login

notebook_login()
```

```
VBox(children=(HTML(value='<center> <img\nsrc=https://huggingface.co/front/assets/huggingface_logo-noborder.sv…
```

Then you need to install Git-LFS to upload your model checkpoints:

In [4]:
```
%%capture
!sudo apt -qq install git-lfs
!git config --global credential.helper store
```

Finally, let's import the libraries we'll be using and define a few convenience functions which we'll use later in the notebook:

In [5]:
```
import numpy as np
import torch
import torch.nn.functional as F
from matplotlib import pyplot as plt
from PIL import Image


def show_images(x):
    """Given a batch of images x, make a grid and convert to PIL"""
    x = x * 0.5 + 0.5  # Map from (-1, 1) back to (0, 1)
    grid = torchvision.utils.make_grid(x)
    grid_im = grid.detach().cpu().permute(1, 2, 0).clip(0, 1) * 255
    grid_im = Image.fromarray(np.array(grid_im).astype(np.uint8))
    return grid_im


def make_grid(images, size=64):
    """Given a list of PIL images, stack them together into a line for easy viewing"""
    output_im = Image.new("RGB", (size * len(images), size))
    for i, im in enumerate(images):
        output_im.paste(im.resize((size, size)), (i * size, 0))
    return output_im


# Mac users may need device = 'mps' (untested)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

OK, we're all set!

Dreambooth: A Sneak Peak at What's to Come

If you've looked at AI-related social media at all in the past few months, you've heard about Stable Diffusion. It's a powerful text-conditioned latent diffusion model (don't worry, we'll learn what all that means). But it has a flaw: it doesn't know what you or I look like unless we're famous enough to have our images plastered around the internet.

Dreambooth let's us create our own model variant with some extra knowledge of a specific face, object or style. The Corridor Crew made an excellent video using this to tell stories with consistent characters, which is a great example of what this technique can do:

https://www.youtube.com/watch?v=W4Mcuh38wyM

In [6]:
```
from IPython.display import YouTubeVideo

YouTubeVideo("W4Mcuh38wyM")
```

Out[6]:

Here's an example using a model trained on 5 photos of a popular children's toy called "Mr Potato Head".

First, we load the pipeline. This will download model weights etc. from the Hub. Since this will download several gigabytes of data for a one-line demo, you are welcome to skip this cell and simply admire the example output!

In [7]:
```
from diffusers import StableDiffusionPipeline

# Check out https://huggingface.co/sd-dreambooth-library for loads of models from the community
model_id = "sd-dreambooth-library/mr-potato-head"

# Load the pipeline
pipe = StableDiffusionPipeline.from_pretrained(model_id, torch_dtype=torch.float16).to(
    device
)
```

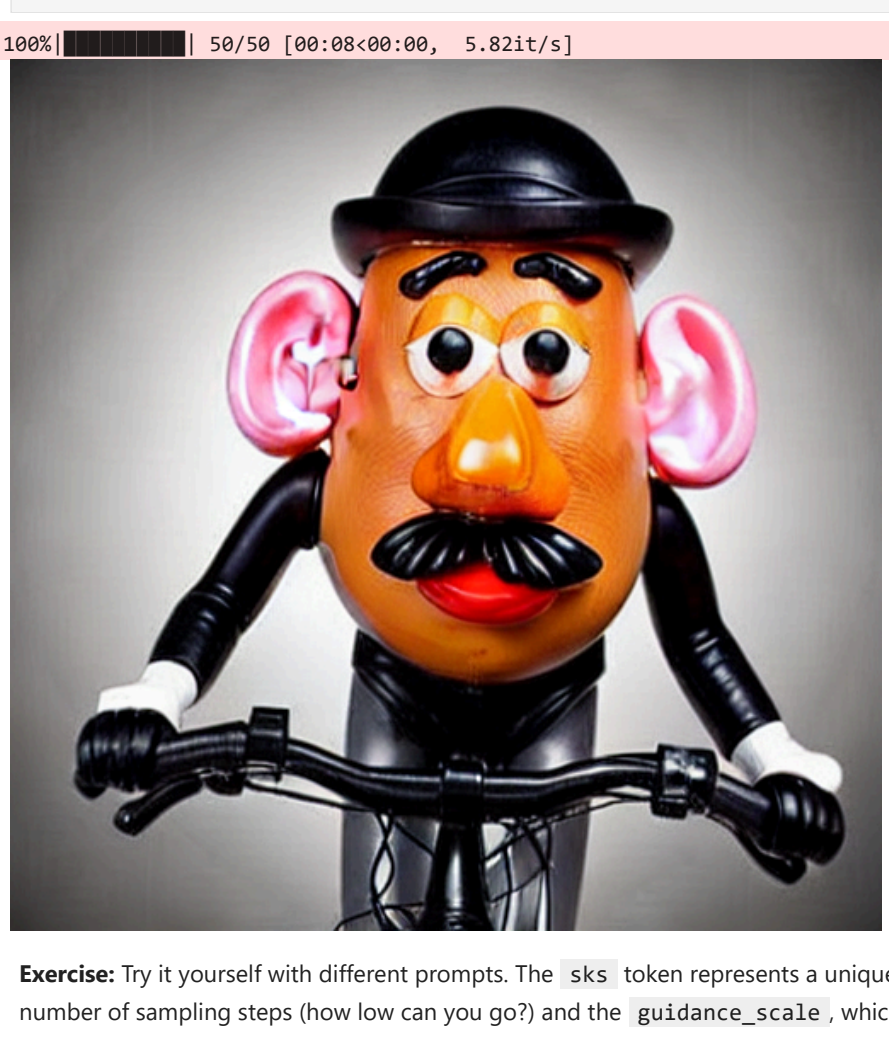
```
Fetching 15 files: 100%|██████████| 15/15 [14:22<00:00, 57.51s/it]
Loading weights: 100%|██████████| 196/196 [00:00<00:00, 400.84it/s, Materializing param=text_model.final_layer_norm.weight]
CLIPTextModel LOAD REPORT from: C:\Users\eraye\.cache\huggingface\hub\models--sd-dreambooth-library--mr-potato-head\snapshots\34246b94acd8b96166e8d93eaded001c380a2d4e\text_encoder
Key                                | Status     |  |
-----------------------------------+------------+--+-
text_model.embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Loading pipeline components...:  14%|█         | 1/7 [00:00<00:03,  1.56it/s]An error occurred while trying to fetch C:\Users\eraye\.cache\huggingface\hub\models--sd-dreambooth-library--mr-p
otato-head\snapshots\34246b94acd8b96166e8d93eaded001c380a2d4e\vae: Error no file named diffusion_pytorch_model.safetensors found in directory C:\Users\eraye\.cache\huggingface\hub\models--sd-
dreambooth-library--mr-potato-head\snapshots\34246b94acd8b96166e8d93eaded001c380a2d4e\vae.
Defaulting to unsafe serialization. Pass `allow_pickle=False` to raise an error instead.
Loading weights: 100%|██████████| 396/396 [00:00<00:00, 403.92it/s, Materializing param=visual_projection.weight]
StableDiffusionSafetyChecker LOAD REPORT from: C:\Users\eraye\.cache\huggingface\hub\models--sd-dreambooth-library--mr-potato-head\snapshots\34246b94acd8b96166e8d93eaded001c380a2d4e\safety_ch
ecker
Key                                              | Status     |  |
-------------------------------------------------+------------+--+-
vision_model.vision_model.embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Loading pipeline components...:  57%|█████     | 4/7 [00:01<00:01,  2.25it/s]An error occurred while trying to fetch C:\Users\eraye\.cache\huggingface\hub\models--sd-dreambooth-library--mr-p
otato-head\snapshots\34246b94acd8b96166e8d93eaded001c380a2d4e\unet: Error no file named diffusion_pytorch_model.safetensors found in directory C:\Users\eraye\.cache\huggingface\hub\models--sd
-dreambooth-library--mr-potato-head\snapshots\34246b94acd8b96166e8d93eaded001c380a2d4e\unet.
Defaulting to unsafe serialization. Pass `allow_pickle=False` to raise an error instead.
Loading pipeline components...: 100%|██████████| 7/7 [00:02<00:00,  3.26it/s]
```

Once the pipeline has finished loading, we can generate images with:

In [11]:
```
prompt = "above a bicycle of sks mr potato head by picasso"
image = pipe(prompt, num_inference_steps=50, guidance_scale=7.5).images[0]
image
```

```
100%|██████████| 50/50 [00:08<00:00,  5.82it/s]
```

Out[11]:



Exercise: Try it yourself with different prompts. The `sks` token represents a unique identifier for the novel concept in this case - what happens if you leave that out? You can also experiment with changing the number of sampling steps (how low can you go?) and the `guidance_scale`, which determines how much the model will try to match the prompt.

There's a lot going on in that magical pipeline! By the end of the course you'll know how it all works. For now, let's take a look at how we can train a diffusion model from scratch.

MVP (Minimum Viable Pipeline)

The core API of 🤗 Diffusers is divided into three main components:

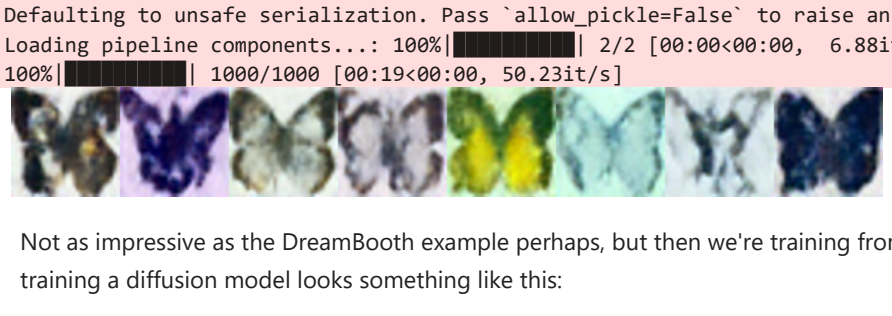
1. **Pipelines**: high-level classes designed to rapidly generate samples from popular trained diffusion models in a user-friendly fashion.
2. **Models**: popular architectures for training new diffusion models, *e.g.* UNet.
3. **Schedulers**: various techniques for generating images from noise during *inference* as well as to generate noisy images for *training*.

Pipelines are great for end-users, but if you're here for this course we assume you want to know what is going on under the hood! So, over the rest of this notebook we're going to build our own pipeline capable of generating small butterfly pictures. Here's the final result in action:

In [12]:
```
from diffusers import DDPMPipeline

# Load the butterfly pipeline
butterfly_pipeline = DDPMPipeline.from_pretrained(
    "johnowhitaker/ddpm-butterflies-32px"
).to(device)

# Create 8 images
images = butterfly_pipeline(batch_size=8).images

# View the result
make_grid(images)
```

```
Fetching 4 files: 100%|██████████| 4/4 [00:16<00:00,  4.12s/it]
Loading pipeline components...:   0%|          | 0/2 [00:00<?, ?it/s]An error occurred while trying to fetch C:\Users\eraye\.cache\huggingface\hub\models--johnowhitaker--ddpm-butterflies-32px
\snapshots\ff99059aed185f6158729b8e9f2286c4b9c177f3: Error no file named diffusion_pytorch_model.safetensors found in directory C:\Users\eraye\.cache\huggingface\hub\models--johnowhitaker--dd
pm-butterflies-32px\snapshots\ff99059aed185f6158729b8e9f2286c4b9c177f3.
Defaulting to unsafe serialization. Pass `allow_pickle=False` to raise an error instead.
Loading pipeline components...: 100%|██████████| 2/2 [00:00<00:00,  6.88it/s]
100%|██████████| 1000/1000 [00:19<00:00, 50.23it/s]
```

Out[12]:



Not as impressive as the DreamBooth example perhaps, but then we're training from scratch with ~0.0001% of the data used to train Stable Diffusion. Speaking of training, recall from the introduction to this unit that training a diffusion model looks something like this:

1. Load in some images from the training data
2. Add noise, in different amounts.
3. Feed the noisy versions of the inputs into the model
4. Evaluate how well the model does at denoising these inputs
5. Use this information to update the model weights, and repeat

We'll explore these steps one by one in the next few sections until we have a complete training loop working, and then we'll explore how to sample from the trained model and how to package everything up into a pipeline for easy sharing. Let's begin with the data...

Step 2: Download a training dataset

For this example, we'll use a dataset of images from the Hugging Face Hub. Specifically, this collection of 1000 butterfly pictures. This is a very small dataset, so we've also included commented out lines for a few larger options. If you'd prefer to use your own collection of images, you can also use the commented-out code example to load in pictures from a folder instead.

In [13]:
```
import torchvision
from datasets import load_dataset
from torchvision import transforms

dataset = load_dataset("huggan/smithsonian_butterflies_subset", split="train")

# Or load images from a local folder
# dataset = load_dataset("imagefolder", data_dir="path/to/folder")

# We'll train on 32-pixel square images, but you can try larger sizes too
image_size = 32
# You can lower your batch size if you're running out of GPU memory
batch_size = 64

# Define data augmentations
preprocess = transforms.Compose(
    [
        transforms.Resize((image_size, image_size)),  # Resize
        transforms.RandomHorizontalFlip(),  # Randomly flip (data augmentation)
        transforms.ToTensor(),  # Convert to tensor (0, 1)
        transforms.Normalize([0.5], [0.5]),  # Map to (-1, 1)
    ]
)


def transform(examples):
    images = [preprocess(image.convert("RGB")) for image in examples["image"]]
    return {"images": images}


dataset.set_transform(transform)

# Create a dataloader from the dataset to serve up the transformed images in batches
train_dataloader = torch.utils.data.DataLoader(
    dataset, batch_size=batch_size, shuffle=True
)
```

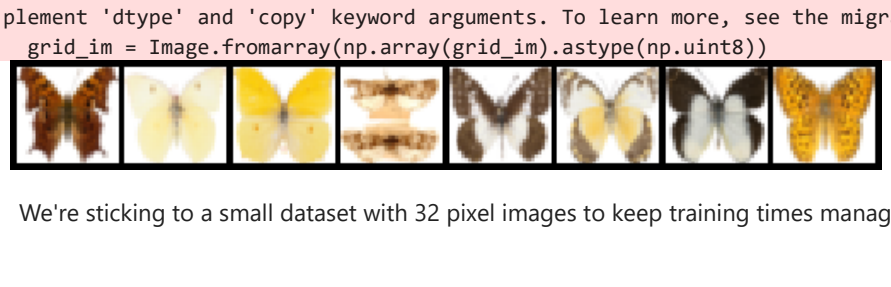
```
Repo card metadata block was not found. Setting CardData to empty.
Generating train split: 100%|██████████| 1000/1000 [00:00<00:00, 2283.45 examples/s]
```

We can grab a batch of images and view some of them like so:

In [16]:
```
xb = next(iter(train_dataloader))["images"].to(device)[:8]
print("X shape:", xb.shape)
show_images(xb).resize((8 * 64, 64), resample=Image.NEAREST)
```

```
X shape: torch.Size([8, 3, 32, 32])
C:\Users\eraye\AppData\Local\Temp\ipykernel_28920\1456380996.py:13: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so passing copy=False failed. __array__ must im
plement 'dtype' and 'copy' keyword arguments. To learn more, see the migration guide https://numpy.org/devdocs/numpy_2_0_migration_guide.html#adapting-to-changes-in-the-copy-keyword
  grid_im = Image.fromarray(np.array(grid_im).astype(np.uint8))
```

Out[16]:



We're sticking to a small dataset with 32 pixel images to keep training times manageable in this notebook.

Step 3: Define the Scheduler

Our plan for training is to take these input images and add noise to them, then feed the noisy images to the model. And during inference, we will use the model predictions to iteratively remove noise. In `diffusers`, these processes are both handled by the **scheduler**.

The noise schedule determines how much noise is added at different timesteps. Here's how we might create a scheduler using the default settings for 'DDPM' training and sampling (based on the paper "Denoising Diffusion Probabilistic Models"):

In [19]:
```
from diffusers import DDPMScheduler

# noise_scheduler = DDPMScheduler(num_train_timesteps=1000)

# The 'cosine' schedule, which may be better for small image sizes:
noise_scheduler = DDPMScheduler(num_train_timesteps=1000, beta_schedule='squaredcos_cap_v2')
```

The DDPM paper describes a corruption process that adds a small amount of noise for every 'timestep'. Given x_{t-1} for some timestep, we can get the next (slightly more noisy) version x_t with:

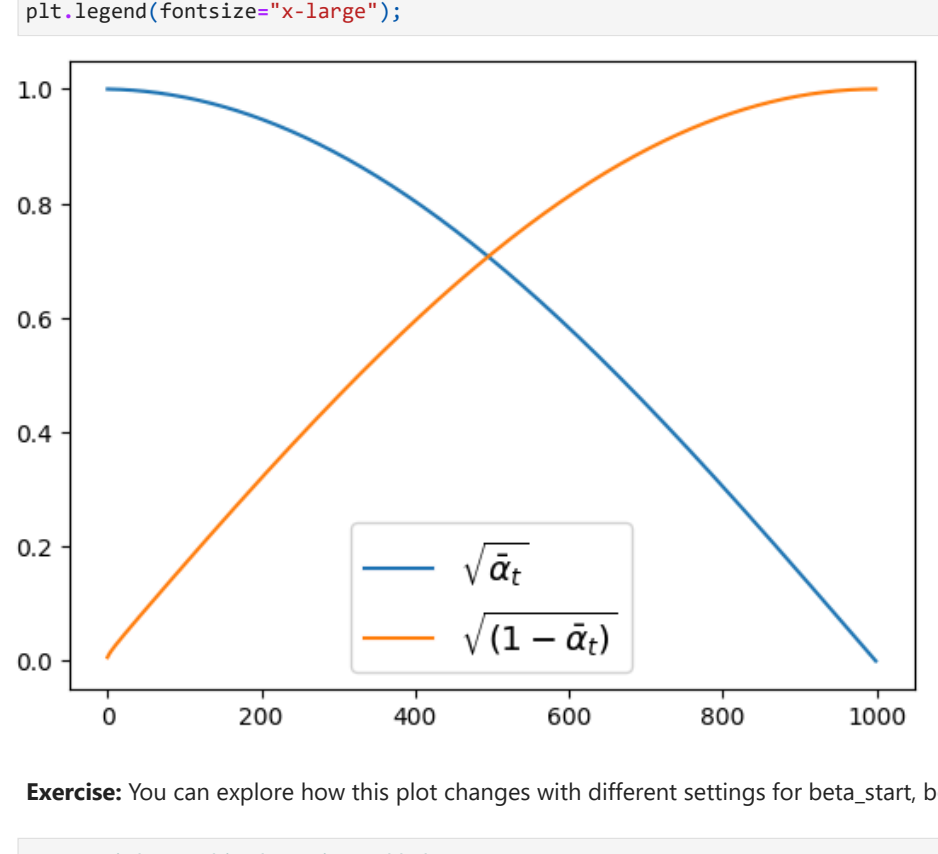
$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

That is, we take x_{t-1}, scale it by $\sqrt{1-\beta_t}$ and add noise scaled by β_t. This β is defined for every t according to some schedule, and determines how much noise is added per timestep. Now, we don't necessarily want to do this operation 500 times to get x_{500} so we have another formula to get x_t for any t given x_0:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar\alpha_t}\mathbf{x}_0, (1-\bar\alpha_t)\mathbf{I}) \text{ where } \bar\alpha_t = \prod_{i=1}^{T}\alpha_i \text{ and } \alpha_i = 1-\beta_i$$

The maths notation always looks scary! Luckily the scheduler handles all that for us. We can plot $\sqrt{\bar\alpha_t}$ (labelled as `sqrt_alpha_prod`) and $\sqrt{(1-\bar\alpha_t)}$ (labelled as `sqrt_one_minus_alpha_prod`) to view how the input (x) and the noise are scaled and mixed across different timesteps:

In [20]:
```
plt.plot(noise_scheduler.alphas_cumprod.cpu() ** 0.5, label=r"${\sqrt{\bar{\alpha}_t}}$")
plt.plot((1 - noise_scheduler.alphas_cumprod.cpu()) ** 0.5, label=r"$\sqrt{(1 - \bar{\alpha}_t)}$")
plt.legend(fontsize="x-large");
```



Exercise: You can explore how this plot changes with different settings for beta_start, beta_end and beta_schedule by swapping in one of the commented-out options here:

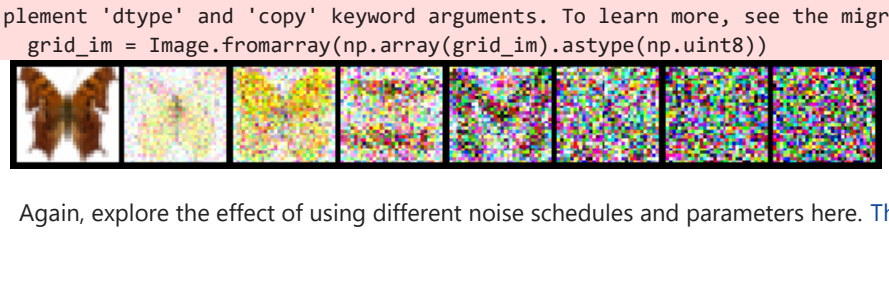
In [ ]:
```
# One with too little noise added:
# noise_scheduler = DDPMScheduler(num_train_timesteps=1000, beta_start=0.001, beta_end=0.004)
# The 'cosine' schedule, which may be better for small image sizes:
# noise_scheduler = DDPMScheduler(num_train_timesteps=1000, beta_schedule='squaredcos_cap_v2')
```

Whichever scheduler you've chosen, we can now use it to add noise in different amounts using the `noise_scheduler.add_noise` function like so:

In [22]:
```
timesteps = torch.linspace(0, 999, 8).long().to(device)
noise = torch.randn_like(xb)
noisy_xb = noise_scheduler.add_noise(xb, noise, timesteps)
print("Noisy X shape", noisy_xb.shape)
show_images(noisy_xb).resize((8 * 64, 64), resample=Image.NEAREST)
```

```
Noisy X shape torch.Size([8, 3, 32, 32])
C:\Users\eraye\AppData\Local\Temp\ipykernel_28920\1456380996.py:13: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so passing copy=False failed. __array__ must im
plement 'dtype' and 'copy' keyword arguments. To learn more, see the migration guide https://numpy.org/devdocs/numpy_2_0_migration_guide.html#adapting-to-changes-in-the-copy-keyword
  grid_im = Image.fromarray(np.array(grid_im).astype(np.uint8))
```

Out[22]:

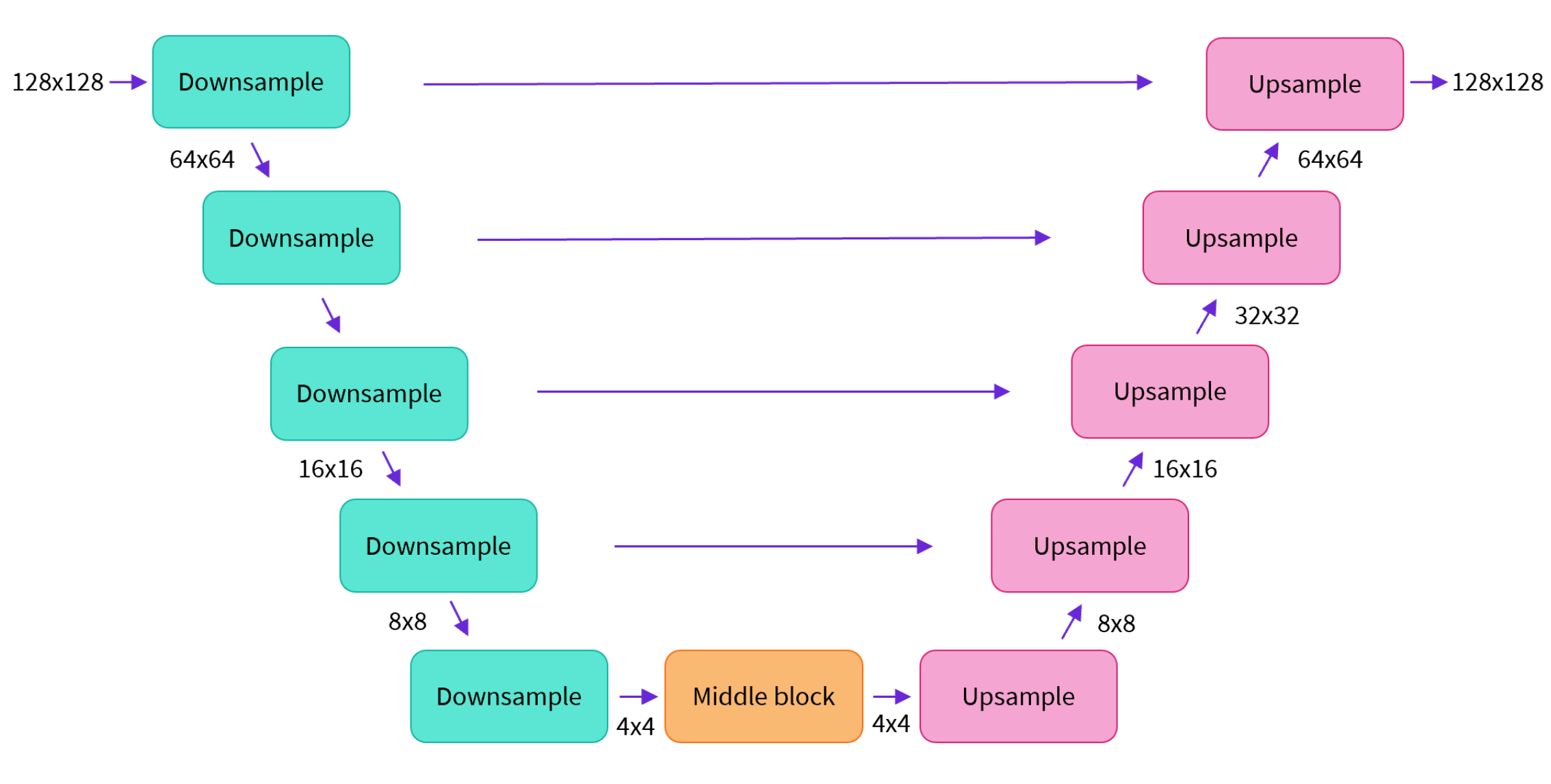


Again, explore the effect of using different noise schedules and parameters here. This video does a great job explaining some of the maths above in more detail, and is a great introduction to some of these concepts.

Step 4: Define the Model

Now we come to the core component: the model itself.

Most diffusion models use architectures that are some variant of a U-net and that's what we'll use here.



In a nutshell:

- the model has the input image go through several blocks of ResNet layers, each of which halves the image size by 2
- then through the same number of blocks that upsample it again.
- there are skip connections linking the features on the downsample path to the corresponding layers in the upsample path.

A key feature of this model is that it predicts images of the same size as the input, which is exactly what we need here.

Diffusers provides us a handy `UNet2DModel` class which creates the desired architecture in PyTorch.

Let's create a U-net for our desired image size. Note that `down_block_types` correspond to the downsampling blocks (green on the diagram above), and `up_block_types` are the upsampling blocks (red on the diagram):

In [23]:
```
from diffusers import UNet2DModel

# Create a model
model = UNet2DModel(
    sample_size=image_size,  # the target image resolution
    in_channels=3,  # the number of input channels, 3 for RGB images
    out_channels=3,  # the number of output channels
    layers_per_block=2,  # how many ResNet layers to use per UNet block
    block_out_channels=(64, 128, 128, 256),  # More channels -> more parameters
    down_block_types=(
        "DownBlock2D",  # a regular ResNet downsampling block
        "DownBlock2D",
        "AttnDownBlock2D",  # a ResNet downsampling block with spatial self-attention
        "AttnDownBlock2D",
    ),
    up_block_types=(
        "AttnUpBlock2D",
        "AttnUpBlock2D",  # a ResNet upsampling block with spatial self-attention
        "UpBlock2D",
        "UpBlock2D",  # a regular ResNet upsampling block
    ),
)
model.to(device);
```

When dealing with higher-resolution inputs you may want to use more down and up-blocks, and keep the attention layers only at the lowest resolution (bottom) layers to reduce memory usage. We'll talk later about how you might experiment to find the best settings for your use-case.

We can check that passing in a batch of data and some random timesteps produces an output the same shape as the input data:

In [24]:
```
with torch.no_grad():
    model_prediction = model(noisy_xb, timesteps).sample
model_prediction.shape
```

Out[24]:
```
torch.Size([8, 3, 32, 32])
```

In the next section we'll see how to train this model.

Step 5: Create a Training Loop

Time to train! Below is a typical optimization loop in PyTorch, where we run through the data batch by batch and update the parameters of our model each step using an optimizer - in this case the AdamW optimizer with a learning rate of 0.0004.

For each batch of data, we

- Sample some random timesteps
- Noise the data accordingly
- Feed the noisy data through the model
- Compare the model predictions with the target (i.e. the noise in this case) using mean squared error as our loss function
- Update the model parameters via `loss.backward()` and `optimizer.step()`

During this process we also log the losses over time for later plotting.

NB: This code takes nearly 10 minutes to run - feel free to skip these two cells and use the pretrained model if you are in a hurry. Alternatively, you can explore how reducing the number of channels in each layer via the model definition above can speed things up.

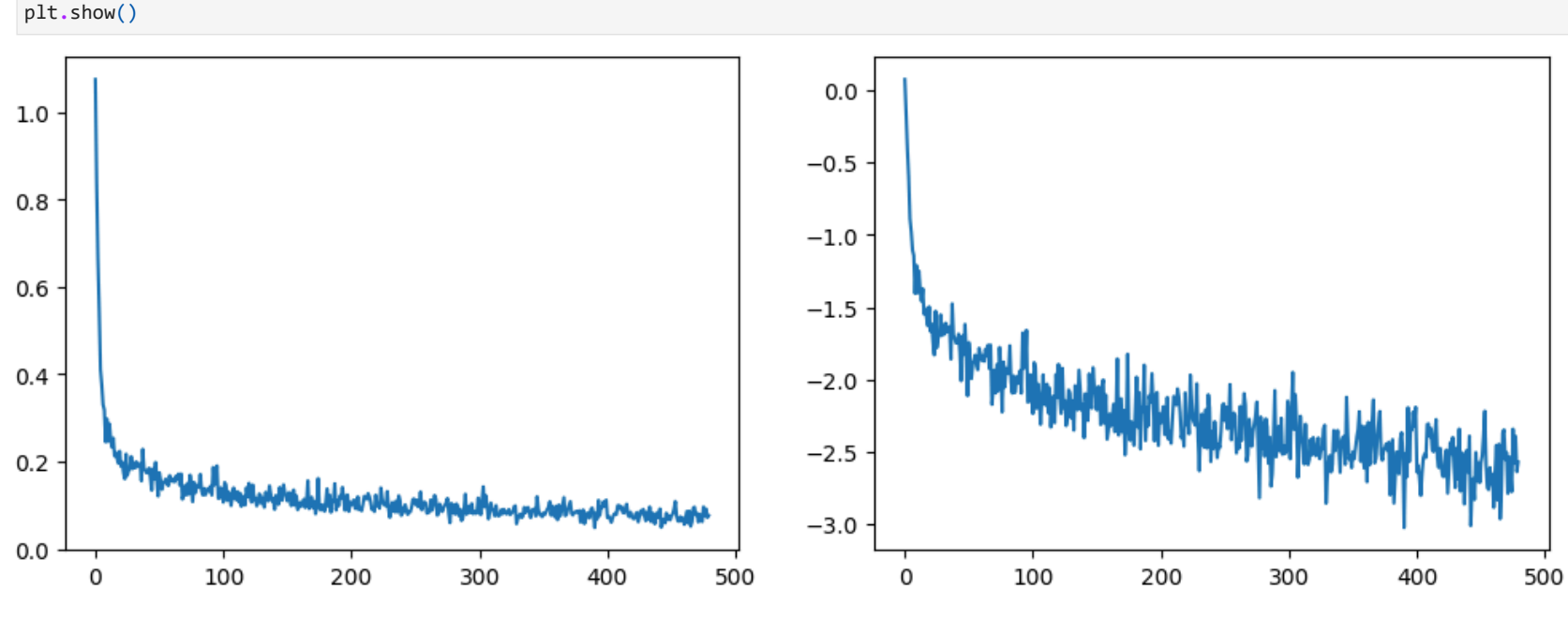
The official diffusers training example trains a larger model on this dataset at higher resolution, and is a good reference for what a less minimal training loop looks like:

In [25]:
```
# Set the noise scheduler
noise_scheduler = DDPMScheduler(
    num_train_timesteps=1000, beta_schedule="squaredcos_cap_v2"
)

# Training loop
optimizer = torch.optim.AdamW(model.parameters(), lr=4e-4)

losses = []

for epoch in range(30):
    for step, batch in enumerate(train_dataloader):
        clean_images = batch["images"].to(device)
        # Sample noise to add to the images
        noise = torch.randn(clean_images.shape).to(clean_images.device)
        bs = clean_images.shape[0]

        # Sample a random timestep for each image
        timesteps = torch.randint(
            0, noise_scheduler.num_train_timesteps, (bs,), device=clean_images.device
        ).long()

        # Add noise to the clean images according to the noise magnitude at each timestep
        noisy_images = noise_scheduler.add_noise(clean_images, noise, timesteps)

        # Get the model prediction
        noise_pred = model(noisy_images, timesteps, return_dict=False)[0]

        # Calculate the loss
        loss = F.mse_loss(noise_pred, noise)
        loss.backward(loss)
        losses.append(loss.item())

        # Update the model parameters with the optimizer
        optimizer.step()
        optimizer.zero_grad()

    if (epoch + 1) % 5 == 0:
        loss_last_epoch = sum(losses[-len(train_dataloader) :]) / len(train_dataloader)
        print(f"Epoch:{epoch+1}, loss: {loss_last_epoch}")
```

```
c:\Users\eraye\AppData\Local\Programs\Python\Python313\Lib\site-packages\diffusers\configuration_utils.py:141: FutureWarning: Accessing config attribute `num_train_timesteps` directly via 'DD
PMScheduler' object attribute is deprecated. Please access 'num_train_timesteps' over 'DDPMScheduler's config object instead, e.g. 'scheduler.config.num_train_timesteps'.
  deprecate("direct config name access", "1.0.0", deprecation_message, standard_warn=False)
Epoch:5, loss: 0.14268582919612527
Epoch:10, loss: 0.11869023600593209
Epoch:15, loss: 0.09936759853735566
Epoch:20, loss: 0.08864026004448533
Epoch:25, loss: 0.08728644717484713
Epoch:30, loss: 0.07715853326953948
```

Plotting the loss, we see that the model rapidly improves initially and then continues to get better at a slower rate (which is more obvious if we use a log scale as shown on the right):

In [26]:
```
fig, axs = plt.subplots(1, 2, figsize=(12, 4))
axs[0].plot(losses)
axs[1].plot(np.log(losses))
plt.show()
```



As an alternative to running the training code above, you can use the model from the pipeline like so:

In [ ]:
```
# Uncomment to instead load the model I trained earlier:
# model = butterfly_pipeline.unet
```

Step 6: Generate Images

How do we get images with this model?

Option 1: Creating a pipeline:

In [27]:
```
from diffusers import DDPMPipeline

image_pipe = DDPMPipeline(unet=model, scheduler=noise_scheduler)
```

In [28]:
```
pipeline_output = image_pipe()
pipeline_output.images[0]
```

```
100%|██████████| 1000/1000 [00:18<00:00, 55.23it/s]
```

Out[28]:

We can save a pipeline to a local folder like so:

In [29]:
```
image_pipe.save_pretrained("my_pipeline")
```

Inspecting the folder contents:

In [47]:
```
import os
print(os.listdir("my_pipeline"))
```

```
['model_index.json', 'scheduler', 'unet']
```

The `scheduler` and `unet` subfolders contain everything needed to re-create those components. For example, inside the `unet` folder you'll find the model weights (`diffusion_pytorch_model.bin`) alongside a config file which specifies the UNet architecture.

In [48]:
```
print(os.listdir("my_pipeline/unet"))
```

```
['config.json', 'diffusion_pytorch_model.safetensors']
```

Together, these files contain everything needed to recreate the pipeline. You can manually upload them to the hub to share the pipeline with others, or check out the code to do this via the API in the next section.

Option 2: Writing a Sampling Loop

If you inspect the forward method of the pipeline you'll be able to see what is happening when we run `image_pipe()`:

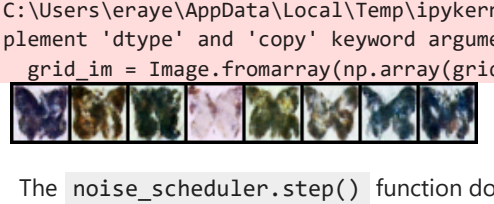
In [ ]:
```
# ??image_pipe.forward
```

We begin with random noise, and run through the scheduler timesteps from most to least noisy, removing a small amount of noise each step based on the model prediction:

In [49]:
```
# Random starting point (8 random images):
sample = torch.randn(8, 3, 32, 32).to(device)

for i, t in enumerate(noise_scheduler.timesteps):

    # Get model pred
    with torch.no_grad():
        residual = model(sample, t).sample

    # Update sample with step
    sample = noise_scheduler.step(residual, t, sample).prev_sample

show_images(sample)
```

```
C:\Users\eraye\AppData\Local\Temp\ipykernel_28920\1456380996.py:13: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so passing copy=False failed. __array__ must im
plement 'dtype' and 'copy' keyword arguments. To learn more, see the migration guide https://numpy.org/devdocs/numpy_2_0_migration_guide.html#adapting-to-changes-in-the-copy-keyword
  grid_im = Image.fromarray(np.array(grid_im).astype(np.uint8))
```

Out[49]:



The `noise_scheduler.step()` function does the maths required to update `sample` appropriately. There are a number of sampling methods - in the next unit we'll see how we can swap in a different sampler to speed up image generation with existing models, and talk more about the theory behind sampling from diffusion models.

Step 7: Push your model to the Hub

In the example above we saved our pipeline to a local folder. To push our model to the Hub, we will need to model repository to push our files to. We'll determine the repository name from the model ID we want to give our model (feel free to replace the `model_name` with your own choice; it just needs to contain your username, which is what the function `get_full_repo_name()` does):

In [50]:
```
from huggingface_hub import get_full_repo_name

model_name = "sd-class-butterflies-32-demo"
hub_model_id = get_full_repo_name(model_name)
hub_model_id
```

Out[50]:
```
'erdoganeray/sd-class-butterflies-32-demo'
```

Next, create a model repository on the 🤗 Hub and push our model:

In [52]:
```
from huggingface_hub import HfApi, create_repo

create_repo(hub_model_id)
api = HfApi()
api.upload_folder(
    folder_path="my_pipeline/scheduler", path_in_repo="", repo_id=hub_model_id
)
api.upload_folder(folder_path="my_pipeline/unet", path_in_repo="", repo_id=hub_model_id)
api.upload_file(
    path_or_fileobj="my_pipeline/model_index.json",
    path_in_repo="model_index.json",
    repo_id=hub_model_id,
)
```

```
Processing Files (1 / 1): 100%|██████████| 74.2MB / 74.2MB, 1.46MB/s
New Data Upload: 100%|██████████| 74.2MB / 74.2MB, 1.46MB/s
```

Out[52]:
```
CommitInfo(commit_url='https://huggingface.co/erdoganeray/sd-class-butterflies-32-demo/commit/35b68994caaaa0998ae9d83e7d778e350c4322c2', commit_message='Upload model_index.json with hugging
face_hub', commit_description='', oid='35b68994caaaa0998ae9d83e7d778e350c4322c2', pr_url=None, repo_url=RepoUrl('https://huggingface.co/erdoganeray/sd-class-butterflies-32-demo', endpoint
='https://huggingface.co', repo_type='model', repo_id='erdoganeray/sd-class-butterflies-32-demo'), pr_revision=None, pr_num=None)
```

The last thing to do is create a nice model card so that our butterfly generator can easily be found on the Hub (feel free to expand and edit the description!):

In [53]:
~~~
from huggingface_hub import ModelCard

content = f"""
---
license: mit
tags:
- pytorch
- diffusers
- unconditional-image-generation
- diffusion-models-class
---

# Model Card for Unit 1 of the [Diffusion Models Class 🧨](https://github.com/huggingface/diffusion-models-class)

This model is a diffusion model for unconditional image generation of cute 🦋.

## Usage

```python
from diffusers import DDPMPipeline

pipeline = DDPMPipeline.from_pretrained('{hub_model_id}')
image = pipeline().images[0]
image
~~~
"""

card = ModelCard(content)
card.push_to_hub(hub_model_id)
```

Out[53]:
```
CommitInfo(commit_url='https://huggingface.co/erdoganeray/sd-class-butterflies-32-demo/commit/9aa3070a17f03cfbacbfc031de69d7132261f9fd', commit_message='Upload README.md with huggingface_hu
b', commit_description='', oid='9aa3070a17f03cfbacbfc031de69d7132261f9fd', pr_url=None, repo_url=RepoUrl('https://huggingface.co/erdoganeray/sd-class-butterflies-32-demo', endpoint='http
s://huggingface.co', repo_type='model', repo_id='erdoganeray/sd-class-butterflies-32-demo'), pr_revision=None, pr_num=None)
```

Now that the model is on the Hub, you can download it from anywhere by using the `from_pretrained()` method of the `DDPMPipeline` as follows"

In [54]:
```
from diffusers import DDPMPipeline

image_pipe = DDPMPipeline.from_pretrained(hub_model_id)
pipeline_output = image_pipe()
pipeline_output.images[0]
```

```
Fetching 4 files: 100%|██████████| 4/4 [00:22<00:00,  5.63s/it]
Loading pipeline components...: 100%|██████████| 2/2 [00:00<00:00, 16.29it/s]
100%|██████████| 1000/1000 [00:35<00:00, 28.16it/s]
```

Out[54]:

Great it works!

Scaling up with 🤗 Accelerate

This notebook was made for learning purposes, and as such I tried to keep the code as minimal and clean as possible. Because of this, we omitted some of the things you might want if you were to try training a larger model on much more data, such as multi-GPU support, logging of progress and example images, gradient checkpointing to support larger batch sizes, automatic uploading of models and so on. Thankfully most of these features are available in the example training script here.

You can download the file like so:

In [ ]:
```
!wget https://github.com/huggingface/diffusers/raw/main/examples/unconditional_image_generation/train_unconditional.py
```

Open up the file and you'll see where the model is defined and what settings are available. I ran the script with the following command:

In [ ]:
```
# Let's give our new model a name for the Hub
model_name = "sd-class-butterflies-64"
hub_model_id = get_full_repo_name(model_name)
hub_model_id
```

Out[ ]:
```
'lewtun/sd-class-butterflies-64'
```

In [ ]:
```
!accelerate launch train_unconditional.py \
  --dataset_name="huggan/smithsonian_butterflies_subset" \
  --resolution=64 \
  --output_dir={model_name} \
  --train_batch_size=32 \
  --num_epochs=50 \
  --gradient_accumulation_steps=1 \
  --learning_rate=1e-4 \
  --lr_warmup_steps=500 \
  --mixed_precision="no"
```

As before, let's push the model to the Hub and create a nice model card (and feel free to edit it as you wish!):

In [ ]:
```
create_repo(hub_model_id)
api = HfApi()
api.upload_folder(
    folder_path=f"{model_name}/scheduler", path_in_repo="", repo_id=hub_model_id
)
api.upload_folder(
    folder_path=f"{model_name}/unet", path_in_repo="", repo_id=hub_model_id
)
api.upload_file(
    path_or_fileobj=f"{model_name}/model_index.json",
    path_in_repo="model_index.json",
    repo_id=hub_model_id,
)

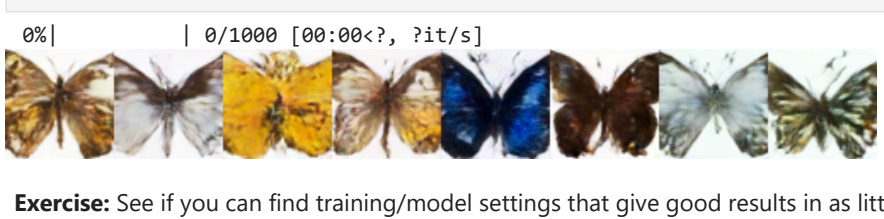
content = f"""
---
license: mit
tags:
- pytorch
- diffusers
- unconditional-image-generation
- diffusion-models-class
---

# Model Card for Unit 1 of the [Diffusion Models Class 🧨](https://github.com/huggingface/diffusion-models-class)

This model is a diffusion model for unconditional image generation of cute 🦋.

## Usage

```python
from diffusers import DDPMPipeline

pipeline = DDPMPipeline.from_pretrained('{hub_model_id}')
image = pipeline().images[0]
image
```
"""

card = ModelCard(content)
card.push_to_hub(hub_model_id)
```

Out[ ]:
```
'https://huggingface.co/lewtun/sd-class-butterflies-64/blob/main/README.md'
```

About 45 minutes later, this is the result:

In [ ]:
```
pipeline = DDPMPipeline.from_pretrained(hub_model_id).to(device)
images = pipeline(batch_size=8).images
make_grid(images)
```

```
  0%|          | 0/1000 [00:00<?, ?it/s]
```

Out[ ]:



Exercise: See if you can find training/model settings that give good results in as little time as possible, and share your findings with the community. Dig around in the script to see if you can understand the code, and ask for clarification on anything that looks confusing.

Avenues for Further Exploration

Hopefully this has given you a taste of what you can do with the 🤗 Diffusers library! Some possible next steps:

- Try training an unconditional diffusion model on a new dataset - bonus points if you create one yourself. You can find some great image datasets for this task in the HugGan organization on the Hub. Just make sure you downsample them if you don't want to wait a very long time for the model to train!
- Try out DreamBooth to create your own customized Stable Diffusion pipeline using either this Space or this notebook
- Modify the training script to explore different UNet hyperparameters (number of layers, channels etc), different noise schedules etc.
- Check out the Diffusion Models from Scratch notebook for a different take on the core ideas we've covered in this unit

Good luck, and stay tuned for Unit 2!
```